

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»

ФАКУЛЬТЕТ АВТОМАТИКИ И ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ
КАФЕДРА ВЫЧИСЛИТЕЛЬНОЙ ТЕХНИКИ

Лабораторная работа №1

По дисциплине «Конструирование и тестирование программного
обеспечения»

По теме «Java. Разработка класса структуры данных»

Вариант №8, 1

Группа: АВТ-341
И.

Преподаватель: Дыминский И.

Студенты: Владимиров А. В.

Шипинский С. А.

Новосибирск 2026

Задание:

Портировать одну из структур данных (СД), приведенных в ermak.cs.nstu.ru/cprog. СД хранит данные произвольного типа:

- ☐ Количество хранимых типов – не менее 2, обертка для 1-3 и 4-12

- ☐ Экземпляр СД хранит объекты только одного типа (например, только целые или только строки)

Набор операций для структуры данных:

- ☐ добавление в конец, получение, вставка и удаление по логическому номеру (индексу). Логический номер – порядковый номер хранимого элемента в соответствии с последовательностью обхода структуры данных (перебора элементов)

- ☐ для исходно упорядоченной (например, «двоичное дерево») вставка с сохранением порядка, получение и удаление по логическому номеру (индексу)

- ☐ Итератор `forEach` – для Java: интерфейс обратного вызова с методом `void toDo(T v)`, итератора получает ссылку на интерфейсный объект (анонимный класс), обходит структуру данных и вызывает метод для каждого выбранного элемента. В Scala – формальный параметр = функция.

- ☐ сортировка, для исходно упорядоченной – балансировка = создание экземпляра структуры данных с равными размерностями компонент (например, сбалансированное дерево, либо список списков со списками нижнего уровня равной длины)

- ☐ сериализация хранимых данных в текстовый (двоичный) файл в формате по выбору – XML, JSON, собственный двоичный (сохранение и загрузка)

Структуру данных реализовать в виде шаблона или реализации интерфейса, работающего с произвольным типом данных (ТД).

Вариант задания:

8. Список с массивом ссылок в каждом элементе

Основной тип хранимых данных:

1. 2D-точка, сравнение по расстоянию от начала координат

UML-диаграммы:

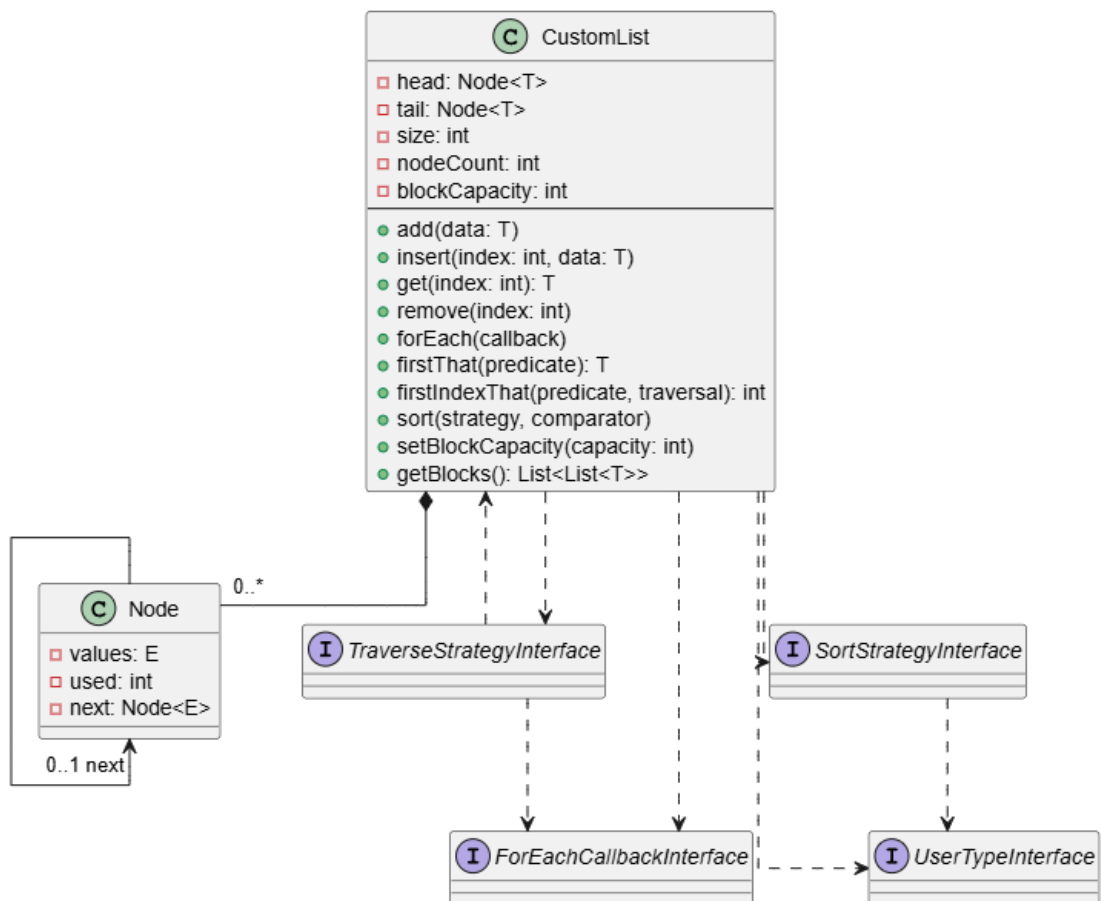


Рисунок 1. Диаграмма классов (базовая структура данных)

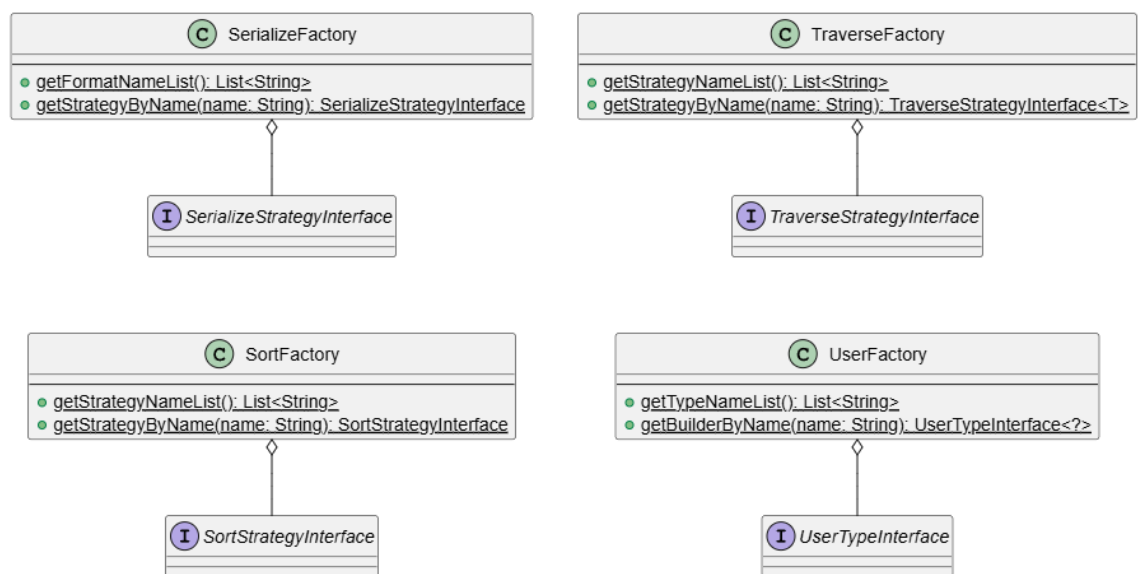


Рисунок 2. Диаграмма классов (Фабрики стратегий)

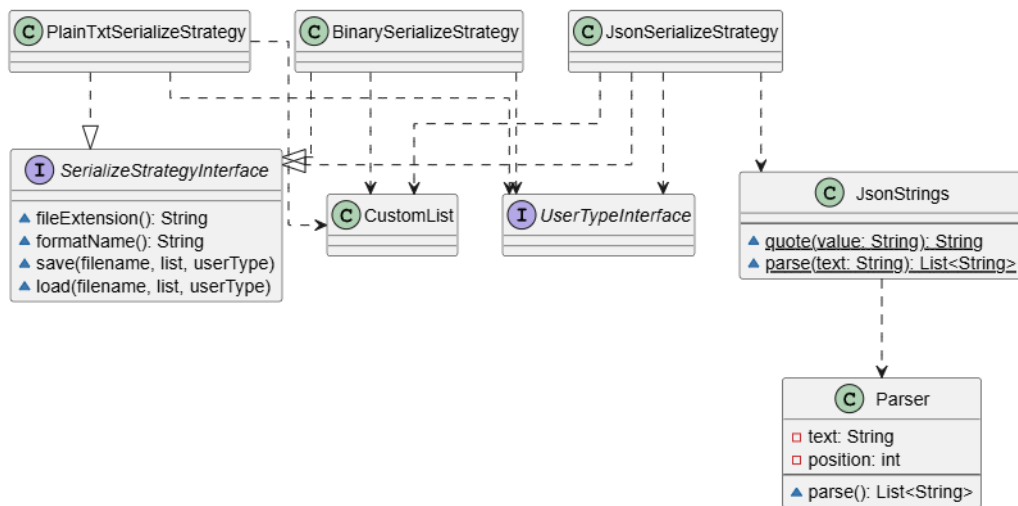


Рисунок 3. Диаграмма классов (Стратегии сериализации)

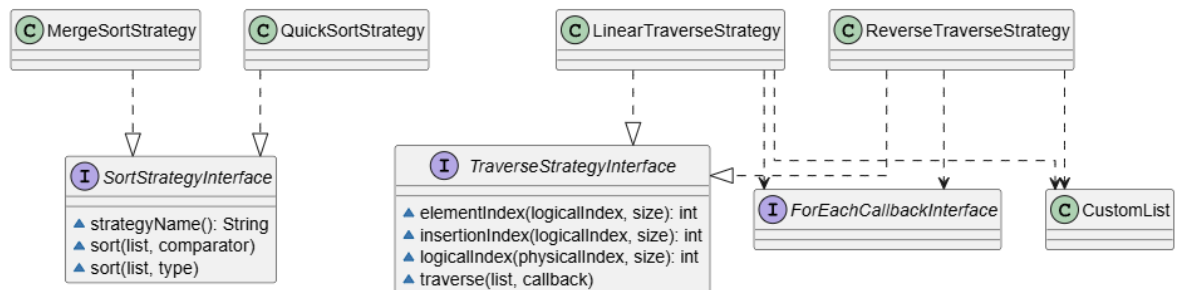


Рисунок 4. Диаграмма классов (Стратегии сортировки)

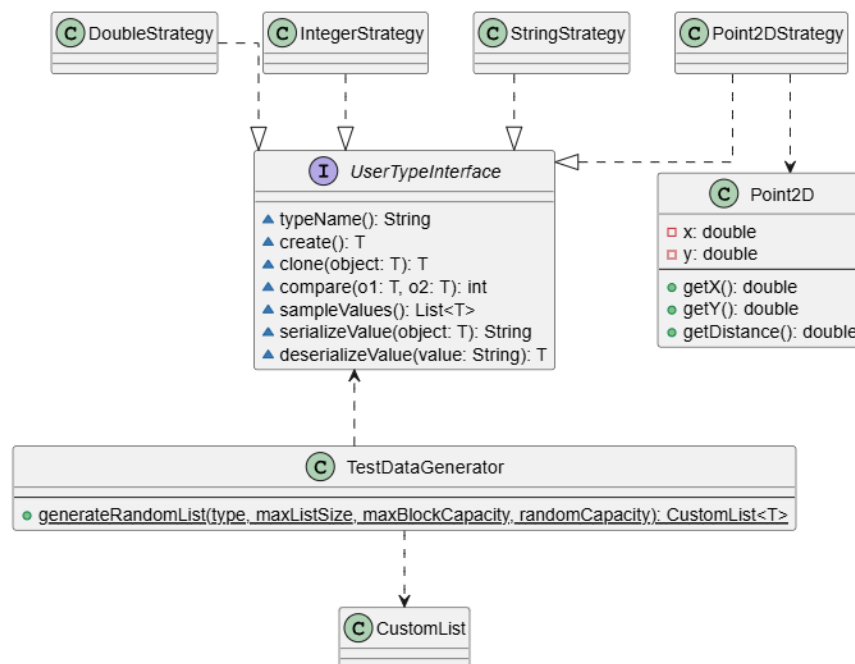


Рисунок 5. Диаграмма классов (Стратегии пользовательских типов)

Ключевые моменты:

Основой является обобщённый класс *CustomList<T>*. Каждый узел списка содержит массив *values*, счётчик заполненных ячеек *used* и ссылку *next* на следующий узел. Список хранит ссылки на первый и последний узлы, общее количество элементов и количество узлов. Размер массива можно изменять, при перегруппировке сохраняются порядок элементов и ссылки на исходные объекты.

В числе функций реализованы добавление элементов, вставка и удаление по индексу, получение значения и очистка списка. При добавлении новый узел создаётся только после заполнения последнего массива. Вставка сопровождается сдвигом элементов вправо, в том числе между соседними узлами. После удаления оставшиеся элементы сдвигаются влево, а опустевший последний узел удаляется. Индексы относятся к общей последовательности элементов и отсчитываются с нуля. Для обработки каждого элемента предусмотрен метод *forEach*. Поиск позволяет получить первый элемент, удовлетворяющий заданному условию и его индекс.

Из типов данных поддерживаются целые, вещественные числа, строки и двумерные точки. Особенности каждого типа описываются через интерфейс *UserTypeInterface<T>*, который определяет создание и копирование объектов, преобразование строк в значения, сравнение, отображение и запись данных. Точки сравниваются по расстоянию от начала координат.

Предусмотрены прямой и обратный обходы. Прямой обход последовательно обрабатывает узлы от начала списка и заполненные ячейки каждого массива. Обратный обход выполняется по временной последовательности ссылок в обратном порядке. Выбранное направление определяет логическую нумерацию элементов и учитывается при получении, вставке, удалении и поиске.

Реализованы два вида сортировок: сортировка слиянием и быстрая сортировка. Для сравнения элементов используется компаратор выбранного типа. Перед сортировкой ссылки на объекты собираются во временный *ArrayList*, после чего упорядоченная последовательность распределяется по узлам. В быстрой сортировке предусмотрено ограничение глубины рекурсии: при его достижении используется сортировка слиянием. Сортировка переставляет ссылки, не создавая копий хранимых объектов.

Список можно сохранять и загружать в форматах *txt*, *bin* и *json*. В файлы записываются значения в прямом порядке независимо от выбранного обхода. При загрузке данные заменяют текущее содержимое списка и распределяются по узлам с установленным размером массива. Перед загрузкой пользователь выбирает соответствующий тип данных.

Графический интерфейс позволяет выбирать тип данных, направление обхода, алгоритм сортировки и формат файла, выполнять операции со списком и генерировать случайные примеры. Визуализация показывает массивы узлов, их ячейки, ссылки на объекты и связи между узлами. Для выбора алгоритмов применяется шаблон «Стратегия», а доступные реализации предоставляются через классы-фабрики. Такое разделение позволяет расширять набор типов и алгоритмов без изменения основной структуры списка.

Интерфейсы и классы:

UserTypeInterface<T> - определяет операции для поддерживаемого типа данных: создание, копирование, сравнение, чтение, запись и строковое представление.

SortStrategyInterface - задаёт общий способ вызова алгоритмов сортировки с использованием компаратора.

TraverseStrategyInterface<T> - определяет порядок обхода списка и преобразование индексов в соответствии с выбранным направлением.

SerializeStrategyInterface - определяет сохранение и загрузку списка, название формата и расширение файла.

ForEachCallbackInterface<T> - задаёт действие, выполняемое для каждого элемента при обходе.

ListEditorPanel.Action - внутренний интерфейс для выполнения действий с общей обработкой.

IntegerStrategy - реализует интерфейс *UserTypeInterface* для целых чисел.

DoubleStrategy - реализует операции над вещественными числами.

StringStrategy - реализует операции над строками.

Point2DStrategy - обеспечивает создание, копирование, преобразование и отображение точек. Сравнение выполняется по расстоянию до начала координат.

Сортировка:

Были реализованы два алгоритма сортировки: сортировка слиянием и быстрая сортировка. Оба реализуют интерфейс *SortStrategyInterface* и используют компаратор, определяющий порядок элементов выбранного типа. Перед сортировкой ссылки на объекты собираются из узлов списка во временный *ArrayList*. После упорядочивания список заново заполняется этими ссылками.

Алгоритм сортировки слиянием реализован в классе *MergeSortStrategy*. Он рекурсивно делит последовательность на две половины, сортирует каждую из них и объединяет результаты. Деление продолжается до получения участков из одного элемента.

```
private <T> void mergeSort(ArrayList<T> list, int l, int r,
                          Comparator<? super T> userType) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(list, l, m, userType);
        mergeSort(list, m + 1, r, userType);
        merge(list, l, m, r, userType);
    }
}
```

Для слияния создаются временные списки левой и правой половин. Их элементы последовательно сравниваются, и меньший записывается в исходный список. После исчерпания одной половины оставшиеся элементы другой копируются без дополнительных сравнений.

```
int i = 0, j = 0, k = l;
while (i < left.size() && j < right.size()) {
    if (userType.compare(left.get(i), right.get(j)) <= 0) {
        list.set(k++, left.get(i++));
    } else {
        list.set(k++, right.get(j++));
    }
}
while (i < left.size()) list.set(k++, left.get(i++));
while (j < right.size()) list.set(k++, right.get(j++));
```

Алгоритм быстрой сортировки реализован в классе *QuickSortStrategy*. В качестве опорного выбирается элемент из середины текущего участка. Два индекса движутся навстречу друг другу: левый пропускает элементы меньше опорного, правый - больше опорного. Найденные элементы, расположенные с неподходящей стороны, меняются местами.

```
T pivot = list.get(low + (high - low) / 2);
int i = low, j = high;
while (i <= j) {
    while (c.compare(list.get(i), pivot) < 0) i++;
    while (c.compare(list.get(j), pivot) > 0) j--;
    if (i <= j) {
        Collections.swap(list, i++, j--);
    }
}
```

После разделения алгоритм рекурсивно обрабатывает полученные участки.

```
quickSort(list, low, j, depth - 1, c);
quickSort(list, i, high, depth - 1, c);
```

Описание пользовательского интерфейса:

Список с массивом ссылок в каждом элементе

Тип данных: 2D-точка Обход итератора: Линейный обход Сериализация: Plain Text

Значение: 3.0 4.0 Логический номер: 0

Вставить в конец Вставить в начало Вставить по номеру Получить Удалить Очистить

Быстрая сортировка Сортировать Обойти Найти

Ячеек в массиве узла: 3 Перегруппировать Объектов: 0 Узлов-массивов: 0

Максимум элементов: 10 Случайный список

Сохранить файл Открыть файл

Рисунок. 2. Внешний вид программы

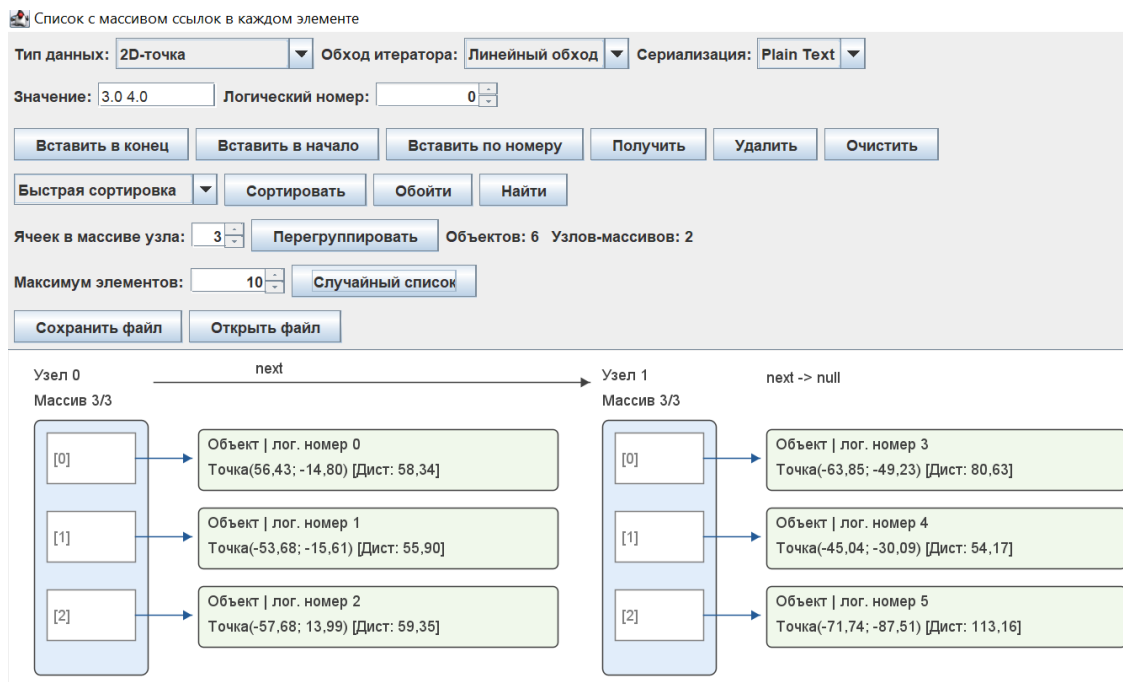


Рисунок 3. Генерация случайного списка

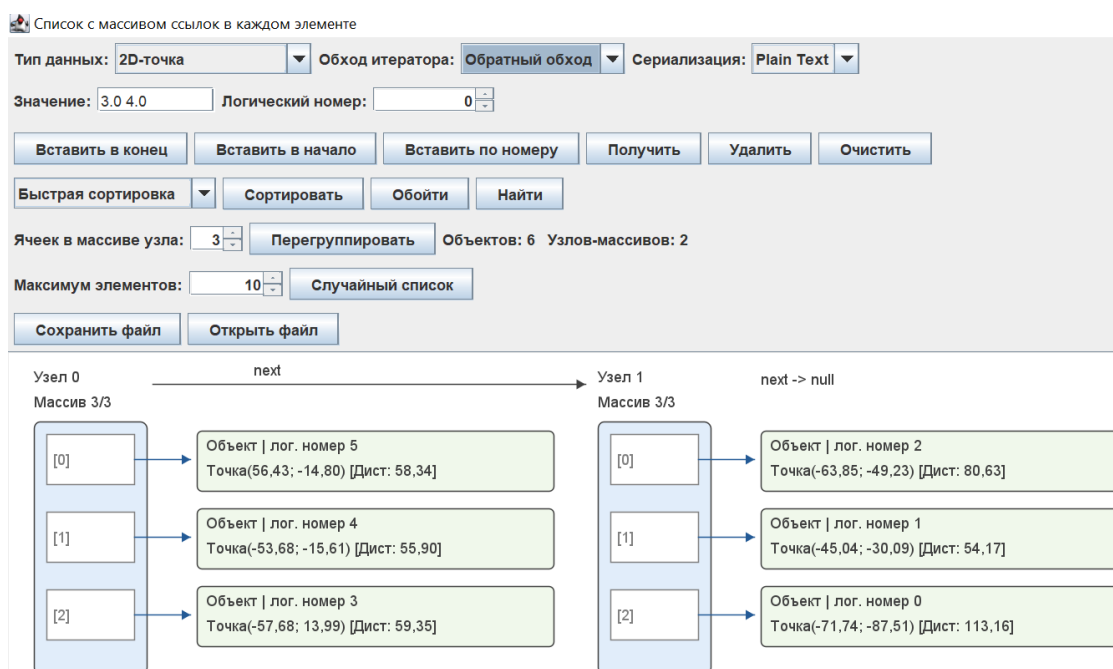


Рисунок 4. Изменение порядка обхода

Вывод:

Была разработана программа для работы со списком, узлы которого содержат массивы ссылок на объекты. Были реализованы основные операции с элементами, прямой и обратный обход, поиск, два алгоритма сортировки, а также сохранение и загрузка данных в

форматах *txt*, *bin* и *json*. Использование интерфейсов и шаблона «Стратегия» обеспечило расширяемость программы, а графический интерфейс позволил наглядно продемонстрировать устройство списка. В результате были закреплены навыки реализации структур данных и применения обобщённых типов.

Полный листинг кода находится в приложении